

TrabFlow Technologies

Motor IA Presupuestos v2

Especificacion para Claude Code

Junio 2026 | Confidencial Interno

1. El Problema: Fallos en la Deteccion de Partidas

1.1 Comportamiento actual (incorrecto)

Las capturas de pantalla muestran que el motor actual devuelve el mensaje 'La IA no detecto partidas' para descripciones validas de trabajo. Esto rompe el flujo completo del instalador.

Descripciones que fallan actualmente:

- "Cambio de termo electrico de 100 litros" -- trabajo de fontaneria estandar
- "Instalacion de red de PEX para vivienda de 2 banos" -- instalacion de tuberias
- "Presupuesto 3 Split aire acondicionado" -- climatizacion
- "Sustitucion completa de fontaneria en vivienda" -- reforma integral fontaneria

1.2 Causa raiz del fallo

El motor actual tiene un unico camino: busca en la Base Maestra → si no encuentra → devuelve error. No tiene fallbacks, no genera sinonimos, no crea partidas temporales. Es un motor de busqueda, no un motor de presupuesto.

PRINCIPIO FUNDAMENTAL: El objetivo del motor NO es encontrar partidas. Es que el instalador SIEMPRE obtenga un presupuesto.

2. Solucion: Motor IA Presupuestos v2

2.1 Filosofia de diseno

- La IA NUNCA responde 'no encontrado' sin haber agotado todos los pasos.
- Cada busqueda fallida genera aprendizaje que reduce futuros fallos.
- El instalador siempre obtiene un presupuesto editable, aunque sea con partidas sugeridas.
- Los precios los decide el instalador. La IA solo estructura y sugiere.

2.2 Flujo completo del motor v2

PASO 1: COMPRENDE LA INTENCION

La IA recibe la descripcion del instalador y ejecuta:

- Extrae el OFICIO principal (fontaneria, electricidad, climatizacion, carpinteria...)
- Identifica las ACTUACIONES solicitadas (instalacion, reparacion, sustitucion, revision...)
- Detecta PARAMETROS clave (numero de unidades, litros, metros, potencia kW, pisos...)
- Genera un PLAN DE TRABAJO estructurado antes de buscar partidas

PASO 2: BUSCA EN BASE MAESTRA (busqueda ampliada)

Busqueda en la tabla trade_catalog_items (catálogo de la empresa) y trade_actuaciones (Base Maestra):

1. Busqueda exacta por nombre normalizado
2. Busqueda semantica por embedding (search_actuaciones_scored)
3. Busqueda por sinonimos tecnicos (hasta 10 por partida)
4. Busqueda por nombres comerciales del sector
5. Busqueda por nombres usados por instaladores en campo

Ejemplos de expansion de busqueda:

Termino original	Expansion (sinonimos + nombres sectoriales)
termo electrico 100L	calentador electrico, termoacumulador, acumulador ACS, boiler 100L, termo Ariston 100L, termo Junkers 100L
red PEX	tuberia multicapa, fontaneria PEX, red de distribucion PEX, instalacion multicapa, tuberias polietileno
split aire acondicionado	unidad split, equipo split, climatizador split, minisplit, bomba de calor inverter
sustitucion fontaneria	reforma fontaneria, renovacion instalacion agua, cambio tuberias, instalacion completa fontaneria

PASO 3: SI NO HAY COINCIDENCIA -- CREA PARTIDA TEMPORAL

Si tras búsqueda ampliada no se encuentran coincidencias suficientes en la Base Maestra, la IA CREA automáticamente una partida temporal con la siguiente estructura:

```
{
  id: 'temp_' + uuid(),
  descripcion: string,           // nombre tecnico inferido
  tipo: 'mano_obra' | 'material',
  cantidad: number,             // inferida del contexto
  horas_estimadas: number | null,
  precio_unitario: 0,           // el instalador lo completa
  subtotal: 0,
  estado: 'nueva_partida_sugerida',
  requiere_precio: true,
  origen: 'ia_generada',
  aviso: 'Nueva partida sugerida por IA -- revisa precio antes de enviar',
  candidatos_catalogo: [],      // alternativas de Base Maestra si las hay
  oficio_detectado: string,
  aprender: true                // flag para guardar en catalogo si el
  instalador acepta
}
```

PASO 4: BUSCA PRODUCTOS EN PROVEEDORES CONFIGURADOS

Para las partidas de tipo 'material' sin precio, el motor busca en proveedores con esta prioridad:

6. Catalogo propio de la empresa (trade_catalog_items con org_id)
7. Historial de presupuestos anteriores de la empresa (trade_quote_items con precio aceptado)
8. OBRAMAT (catalogo PDF ya cargado, disponible para demo)
9. Otros proveedores configurados en trade_mayoristas

Si encuentra producto en proveedor, la partida temporal incluye:

- nombre_producto: nombre exacto del producto del proveedor
- precio_referencia: precio orientativo del proveedor
- proveedor: nombre del proveedor
- referencia: referencia del proveedor si disponible
- url_producto: URL si disponible

PASO 5: MARCA, MUESTRA Y APRENDE

El wizard muestra las partidas con el estado visual correcto:

Estado de partida	Tratamiento visual
Encontrada en Base Maestra	Fondo blanco, precio pre-rellenado si existe en catalogo empresa
Nueva partida sugerida (IA)	Fondo amarillo/gold, badge 'Sugerida por IA', campo precio

	vacio con focus
Producto encontrado en proveedor	Badge 'OBRAMAT' o proveedor, precio de referencia visible pero editable
Sin precio en catalogo	Aviso naranja actual -- comportamiento ya existente mantenido

Cuando el instalador GUARDA el presupuesto:

- Si acepta una 'nueva partida sugerida': se ejecuta INSERT en trade_catalog_items con la descripcion, oficio y precio asignado por el instalador.
- Se ejecuta el RPC update_actuacion_learned / insert_actuacion_learned existente para actualizar usage_count y precio aprendido.
- Las partidas con aprender: true y precio > 0 se guardan automaticamente en el catalogo de la empresa.

3. Cambios Tecnicos en trade-voice-to-quote

3.1 System prompt actualizado

El system prompt de la edge function trade-voice-to-quote debe reemplazarse por la siguiente logica:

```
// SYSTEM PROMPT MOTOR IA PRESUPUESTOS v2
// Archivo: supabase/functions/trade-voice-to-quote/prompts.ts

export const SYSTEM_PROMPT_MOTOR_IA = `
Eres el motor de inteligencia de TrabFlow.
Tu objetivo es que el instalador SIEMPRE obtenga un presupuesto.

PROCESO OBLIGATORIO (en orden, sin saltar pasos):

1. COMPRENDE LA INTENCION:
  - Extrae: oficio, actuaciones, parametros clave
  - Genera un plan de trabajo mental antes de buscar partidas

2. PARA CADA ACTUACION del plan:
  a) Busca en el catalogo proporcionado (busqueda fuzzy + sinonimos)
  b) Si encuentras: usa esa partida
  c) Si NO encuentras: crea partida_sugerida con estado:
  'nueva_partida_sugerida'

3. EXPANSION DE BUSQUEDA (si paso 2a falla):
  - Genera hasta 10 sinonimos tecnicos
  - Genera nombres comerciales del sector
  - Genera nombres usados por instaladores en campo
  - Busca con cada uno

4. NUNCA devuelvas partidas: [] si la descripcion tiene contenido.
  Si no hay nada en catalogo → crea partidas sugeridas.

5. ESTRUCTURA DE RESPUESTA OBLIGATORIA:
  {
    oficio_detectado: string,
    plan_trabajo: string[],
    partidas: PartidaPresupuesto[]
  }

Donde PartidaPresupuesto tiene campo estado:
'catalogo'          → encontrada en Base Maestra
'nueva_partida_sugerida' → creada por IA, pendiente de precio
'proveedor'        → encontrada en catalogo de proveedor
`;
```

3.2 Logica de busqueda en la edge function

El flujo interno de trade-voice-to-quote debe implementar:

```
// trade-voice-to-quote/index.ts -- logica de busqueda ampliada
```

```

async function buscarPartidaConFallback(
  descripcion: string,
  orgId: string,
  supabase: SupabaseClient
): Promise<PartidaPresupuesto> {

  // NIVEL 1: Búsqueda semantica en Base Maestra
  const embedding = await generarEmbedding(descripcion);
  const { data: resultados } = await supabase.rpc('search_actuaciones_scored',
  {
    query_embedding: embedding,
    match_threshold: 0.75,
    match_count: 5,
    org_id_filter: orgId
  });

  if (resultados?.length > 0) {
    return mapActuacionToPartida(resultados[0], 'catalogo');
  }

  // NIVEL 2: Búsqueda con sinonimos generados por IA
  const sinonimos = await generarSinonimos(descripcion); // hasta 10
  for (const sinonimo of sinonimos) {
    const embSin = await generarEmbedding(sinonimo);
    const { data: resSin } = await supabase.rpc('search_actuaciones_scored', {
      query_embedding: embSin,
      match_threshold: 0.80,
      match_count: 3,
      org_id_filter: orgId
    });
    if (resSin?.length > 0) {
      return mapActuacionToPartida(resSin[0], 'catalogo');
    }
  }

  // NIVEL 3: Busca en catalogo de proveedores
  const productoProv = await buscarEnProveedores(descripcion, orgId, supabase);
  if (productoProv) {
    return crearPartidaConProveedor(descripcion, productoProv);
  }

  // NIVEL 4: Crea partida temporal sugerida por IA
  return crearPartidaSugerida(descripcion);
}

```

3.3 Nueva tabla: trade_partidas_aprendidas

Para persistir el aprendizaje de las nuevas partidas sugeridas por la IA que el instalador acepta:

```

-- Migracion SQL: nueva tabla de aprendizaje de partidas
CREATE TABLE trade_partidas_aprendidas (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID REFERENCES trade_organizations(id) ON DELETE CASCADE,
  descripcion TEXT NOT NULL,
  oficio      TEXT,

```

```

    tipo          TEXT CHECK (tipo IN ('mano_obra', 'material')),
    precio_aceptado NUMERIC(10,2),
    proveedor     TEXT,
    referencia    TEXT,
    veces_usada   INTEGER DEFAULT 1,
    primera_vez   TIMESTAMPTZ DEFAULT now(),
    ultima_vez    TIMESTAMPTZ DEFAULT now(),
    CONSTRAINT uq_org_descripcion UNIQUE (org_id, descripcion)
);

-- RPC para guardar o actualizar partida aprendida
CREATE OR REPLACE FUNCTION upsert_partida_aprendida(
    p_org_id UUID,
    p_descripcion TEXT,
    p_oficio TEXT,
    p_tipo TEXT,
    p_precio NUMERIC
) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO trade_partidas_aprendidas
        (org_id, descripcion, oficio, tipo, precio_aceptado)
    VALUES (p_org_id, p_descripcion, p_oficio, p_tipo, p_precio)
    ON CONFLICT (org_id, descripcion) DO UPDATE SET
        precio_aceptado = EXCLUDED.precio_aceptado,
        veces_usada = trade_partidas_aprendidas.veces_usada + 1,
        ultima_vez = now();
END;
$$;

```

4. Cambios en el Frontend (Wizard Paso 4)

4.1 Componente ScreenWizardPresupuesto -- Paso 4

El paso 4 (Revisa el presupuesto) debe diferenciar visualmente los tres estados de partida:

Tipo de partida	Cambio visual requerido
catalogo	Sin cambio. Comportamiento actual (fondo blanco, precio rellenado si existe).
nueva_partida_sugerida	Fondo amarillo suave (#FFF8E1). Badge naranja 'Nueva -- revisa precio'. Precio en 0, campo con focus rojo hasta que el usuario lo rellene.
proveedor	Fondo azul muy claro (#E3F0FB). Badge azul con nombre del proveedor. Precio de referencia visible en gris, campo editable encima.

4.2 Notificacion mejorada en Paso 2

El mensaje de error actual ('La IA no detecto partidas -- revisa la descripcion') debe eliminarse. En su lugar:

- Si la IA genera partidas sugeridas: mostrar notificacion verde 'Partidas generadas automaticamente -- revisa precios antes de continuar'
- Si la descripcion esta completamente vacia o es ininteligible: mostrar sugerencia de ejemplos validos
- Nunca bloquear el flujo. El instalador puede continuar siempre al Paso 3.

4.3 Guardado con aprendizaje

Al ejecutar el guardado final del presupuesto, el frontend debe:

10. Identificar todas las partidas con estado 'nueva_partida_sugerida' y precio > 0
11. Por cada una, llamar al RPC `upsert_partida_aprendida` con `org_id`, `descripcion`, `oficio`, `tipo`, `precio`
12. Mostrar al instalador: 'X nuevas partidas guardadas en tu catalogo para futuros presupuestos'

5. Casos de Prueba Obligatorios

Antes de considerar el motor v2 funcional, estos casos deben generar al menos 3 partidas cada uno:

Descripcion de entrada	Resultado esperado (minimo)
Cambio de termo electrico de 100 litros	Partida material (termo 100L) + Partida mano obra (desmontaje) + Partida mano obra (instalacion)
Instalacion de red de PEX para vivienda de 2 banos	Material tuberia PEX + Accesorios + Mano obra instalacion + Prueba estanqueidad
Presupuesto 3 Split aire acondicionado	3x unidades split (material) + Mano obra instalacion por unidad + Carga gas refrigerante
Sustitucion completa de fontaneria en vivienda	Desmontaje instalacion existente + Tuberias + Griferia + Mano obra + Prueba presion
Instalar punto de carga coche electrico wallbox 22kW	Wallbox material + Cable derivacion + Magnetotermico + Diferencial + Mano obra

CRITERIO DE ACEPTACION: Ningun caso de prueba debe devolver partidas: []. Todos deben generar al menos partidas sugeridas por IA si no hay coincidencia en Base Maestra.

6. Plan de Implementacion

Tarea 1 (2-3h): Actualizar system prompt de trade-voice-to-quote

- Sustituir el system prompt actual por el definido en seccion 3.1
- Anadir la logica de generacion de sinonimos como herramienta del modelo
- Anadir campo estado en el schema JSON de respuesta
- Test local con los 5 casos de prueba de la seccion 5

Tarea 2 (3-4h): Implementar buscarPartidaConFallback

- Implementar la funcion segun el pseudocodigo de seccion 3.2
- Nivel 1: busqueda semantica (ya existe, revisar threshold 0.75)
- Nivel 2: expansion con sinonimos generados por IA
- Nivel 3: busqueda en trade_catalog_items de proveedores
- Nivel 4: creacion de partida sugerida (nunca devolver array vacio)

Tarea 3 (1h): Migracion SQL

- Crear tabla trade_partidas_aprendidas segun seccion 3.3
- Crear RPC upsert_partida_aprendida
- Aplicar migracion en Supabase prod

Tarea 4 (2-3h): Actualizar frontend Paso 4 del wizard

- Anadir estilos visuales segun tabla de seccion 4.1
- Badge 'Nueva -- revisa precio' para partidas sugeridas por IA
- Eliminar notificacion de error actual en Paso 2
- Anadir notificacion verde cuando se generan partidas automaticamente

Tarea 5 (1h): Implementar guardado con aprendizaje

- En el handler de guardado del wizard, extraer partidas con estado 'nueva_partida_sugerida'
- Llamar a upsert_partida_aprendida para cada una con precio > 0
- Toast de confirmacion al instalador

TIEMPO TOTAL ESTIMADO: 9-12 horas de desarrollo. Impacto: elimina el error 'La IA no detecto partidas' en el 100% de los casos con descripcion valida.

7. Notas de Arquitectura

Principios que NO cambian con esta mejora:

- La IA NO asigna precios nunca. Las partidas sugeridas salen con precio: 0.
- El instalador siempre puede editar, eliminar o anadir partidas en el Paso 4.
- El catalogo de la empresa es privado por org_id. RLS activo.
- Los precios aprendidos son por empresa (org_id), no globales.
- El motor no impone proveedores. Muestra opciones. El instalador elige.

Limitaciones conocidas de esta version:

- La busqueda en OBRAMAT requiere que el catalogo PDF este cargado en trade_supplier_products (tabla ya disenada en analisis de proveedor).
- La generacion de sinonimos anade 1 llamada adicional a Claude Haiku por partida no encontrada. Coste estimado: +0.001 EUR por partida sin coincidencia.
- La tabla trade_partidas_aprendidas no sustituye a la Base Maestra. Es el catalogo especifico de la empresa. A futuro se puede promover a Base Maestra global si varias empresas aprenden la misma partida.

Este documento define los cambios minimos para resolver el problema critico de fallos en deteccion de partidas. Entregar a Claude Code como tarea autocontenida.